

Product Requirements Document (PRD)

Project Information

- Project Title:** AI Fleet Route Optimizer
- Domain / Track:** Logistics & Supply Chain | GenAI + Full Stack
- Authors:**
 - Manthan Nimodiya
 - Abhayraj Jaiswal
- Tech Stack:**
 - Frontend:* React, Next.js, React Query, Tailwind CSS
 - Backend & Database:* FastAPI (Python), PostgreSQL, DBMS, Graph/Optimization Algorithms
 - AI / Orchestration:* LangGraph, Multi-Agent Architecture, RAG

Product Goal

Build an enterprise-grade AI solution for Fleet Route Optimization that addresses critical operational efficiency, routing complexity (Vehicle Routing Problem with Time Windows), and dispatch automation challenges in the Logistics sector. The system combines deterministic graph algorithms (DSA) with multi-agent generative AI and RAG to dynamically optimize multi-stop delivery routes, enforce strict operational/payload constraints, and empower dispatchers with an intelligent, context-aware operational copilot.

Feature Priorities

FEATURE	PRIORITY	DESCRIPTION
Role-Based Authentication & Authorization	MUST	Secure JWT-based access for Admins, Fleet Managers, Dispatchers, and Drivers.
Fleet & Asset Management (CRUD)	MUST	Complete CRUD operations for vehicles, drivers, hubs/depots, shipments, and maintenance status.
Deterministic Route Optimization Engine	MUST	DSA-based multi-vehicle routing solver with capacity, time windows, and priority constraints.
Dynamic Rerouting & Traffic Adaptation	MUST	Real-time recalculation of active routes upon ad-hoc delivery insertions, cancellations, or delays.
Multi-Agent Operational Copilot (LangGraph)	MUST	Multi-agent AI system orchestrating routing analysis, constraint verification, and dispatcher interactions.
Retrieval-Augmented Generation (RAG)	MUST	Enterprise SOP, safety regulation, and knowledge base grounding with zero hallucination.
Interactive Map & Real-Time Tracking UI	MUST	Interactive route visualization, stop ordering, geofencing, and telemetry inspection built with Next.js & Tailwind.
Operational Analytics & KPI Dashboard	SHOULD	Metrics tracking total fuel consumption, route efficiency, SLA breach rate, turnaround time, and cost savings.
Automated Dispatch & Driver Notification	SHOULD	Real-time delivery manifests and stop-by-stop dispatch notifications pushed to drivers.
Anomaly & SLA Breach Alerts	COULD	Proactive alerts for route deviations, excessive dwell time, and impending delivery window violations.
Historical Simulation & Scenario Planning	COULD	"What-if" scenario simulation tool to forecast fleet requirements based on historical delivery volumes.

User Stories

US-001: User Authentication & Role-Based Access Control

As a Fleet Operations Administrator,
I want to authenticate securely and assign role-specific permissions (Admin, Dispatcher, Driver),
so that sensitive logistics data, fleet assets, and operational dispatches are access-controlled.

Acceptance criteria

- Secure login and registration with hashed credentials (bcrypt) and signed JWT tokens.
- Role-based route guards and API middleware in FastAPI and Next.js.
- Distinct view permissions: Drivers see assigned trips only; Dispatchers manage routes and live trips; Admins access full configuration and analytics.
- Session expiration and secure token refresh mechanisms implemented.

US-002: Fleet, Driver, and Depot Management (CRUD)

As a Fleet Manager,
I want to create, read, update, and manage vehicles, drivers, customer delivery locations, and distribution hubs,
so that the routing engine has accurate, up-to-date operational constraints.

Acceptance criteria

- Comprehensive CRUD interfaces for vehicles (payload capacity, fuel efficiency, vehicle type, current status).
 - Comprehensive CRUD interfaces for drivers (shifts, licenses, driving-hour restrictions, current assignment).
 - Geocoded depot and delivery waypoint management with latitude, longitude, and contact metadata.
 - Data persistence and relational integrity enforced via PostgreSQL schemas with appropriate indexing.
-

US-003: Deterministic Fleet Route Optimization (DSA & VRPTW)

As a Logistics Dispatcher,
I want to submit a batch of delivery orders and have the system calculate the globally optimal route for my available fleet,
so that fuel consumption, vehicle mileage, and operational transit time are minimized while respecting capacity and time windows.

Acceptance criteria

- Graph-based solver handles multi-depot, multi-vehicle routing with capacity constraints and customer time windows (VRPTW).
 - Produces ordered sequence of waypoints, estimated departure/arrival timestamps, and total route distance/cost.
 - Enforces strict hard constraints (vehicle payload capacity cannot be exceeded; delivery within promised customer time window).
 - Computes optimized routes in under 5 seconds for standard batches (up to 50 stops).
-

US-004: Interactive Route Visualizer & Dispatch Workspace

As a Logistics Dispatcher,
I want to inspect proposed routes on an interactive map with step-by-step waypoint sequences and metrics,
so that I can review, adjust, and approve dispatch schedules intuitively.

Acceptance criteria

- Next.js and Tailwind CSS powered responsive UI with interactive map rendering (polylines, color-coded vehicle paths, depot & stop pins).
 - Fast state updates and caching powered by React Query with optimistic UI updates.
 - Visual indicators for delivery time windows, ETA countdowns, and vehicle load percentages.
 - Manual drag-and-drop stop reordering with immediate recalculation of ETAs and route viability.
-

US-005: Multi-Agent AI Dispatch Copilot (LangGraph)

As an Operations Dispatcher,
I want to interact with an AI dispatch assistant using natural language to solve routing exceptions (e.g., "Vehicle 4 broke down, reassign remaining stops with minimal delay"),
so that operational disruptions can be diagnosed and resolved autonomously without manual recalculations.

Acceptance criteria

- Multi-agent system built using LangGraph featuring distinct specialized agents:
 - *Router Agent*: Classifies incoming dispatcher intent and delegates tasks.
 - *Optimization Agent*: Interacts with the backend solver to trigger re-routing calculations.
 - *Policy & Compliance Agent*: Validates actions against labor laws, vehicle limits, and delivery agreements.
 - Multi-turn conversation state maintained with clear execution traces.
 - AI provides actionable proposals with structured confirmation buttons before applying destructive state changes.
-

US-006: RAG-Powered Knowledge & Compliance Engine

As an Operations or Safety Manager,
I want the AI system to retrieve exact enterprise policies, regional transport regulations, and driver SOPs using RAG,
so that decisions and queries are grounded in verified documentation with zero hallucination.

Acceptance criteria

- Vectorized enterprise knowledge base grounded in the RAG architecture (*Lewis et al., 2020*) and QA benchmark principles (*WikiQA*).
 - Semantic search returns relevant policy excerpts, hazardous materials handling rules, and driver rest-break mandates.
 - Every AI response references source document citations and confidence scores.
 - Fallback guardrail triggers when similarity threshold is not met ("Policy not found in verified knowledge base").
-

US-007: Operational Analytics & KPI Reporting

As a Logistics Executive,
I want an executive analytics dashboard displaying key operational metrics,
so that I can track fleet efficiency, delivery SLA compliance, and cost reductions over time.

Acceptance criteria

- Aggregated metrics: On-Time In-Full (OTIF) delivery rate, average transit duration, fuel burn index, cost per ton-kilometer.
 - Filterable by date range, vehicle type, driver, regional hub, and route difficulty.
 - Interactive charts rendered cleanly with smooth responsive loading states.
 - Export functionality for operational audits (CSV / PDF summary).
-

US-008: Dynamic In-Transit Rerouting & Exception Handling

As a Driver or Dispatcher,
I want to mark stops as completed, skipped, or failed in real-time,
so that downstream stops are dynamically updated and remaining drivers receive recalculated routes.

Acceptance criteria

- Real-time status update API endpoint (ARRIVED , COMPLETED , FAILED , DELAYED).
- Dynamic recalculation updates ETAs for remaining downstream stops.
- Audit log records all route modifications, timestamped state changes, and reasons for deviation.

Product States

The lifecycle of an optimization and dispatch cycle transitions through the following finite state machine:

UNASSIGNED_ORDERS → CLUSTERED → OPTIMIZING → ROUTE_PROPOSED → DISPATCHED → IN_TRANSIT → COMPLETED

State Definitions:

- UNASSIGNED_ORDERS : Delivery shipments are logged in the database without an assigned vehicle or schedule.
- CLUSTERED : Orders are geographically and temporally grouped by destination hub and delivery time window.
- OPTIMIZING : Deterministic VRPTW algorithm and LangGraph agents compute optimal vehicle-to-stop assignments.
- ROUTE_PROPOSED : Computed plan is staged for dispatcher review with ETAs, payload distribution, and cost estimates.
- DISPATCHED : Dispatcher approves the route; digital manifest is locked and pushed to the assigned driver.
- IN_TRANSIT : Vehicle is active on route; waypoints are sequentially serviced with live telemetry tracking.
- COMPLETED : All waypoints have been resolved (DELIVERED or RETURNED), and trip summary is archived for analytics.

Failure & Exception States:

- OPTIMIZATION_FAILED : Solver cannot find a feasible route satisfying all hard constraints (e.g., total volume exceeds total fleet capacity). System highlights violated constraints and prompts the dispatcher.
- ROUTE_BLOCKED : Critical exception encountered during execution (vehicle breakdown, severe road closure). Triggers dynamic re-optimization via the LangGraph AI copilot.
- SLA_BREACHED : Actual delivery timestamp exceeds committed delivery window; flagged automatically for audit.

Product Principles

- Deterministic Safety Before Stochastic Generation:** Core routing, payload capacity constraints, and physical road feasibility are computed by deterministic algorithms and verified database constraints. LLMs assist in orchestration, analysis, and reasoning—never guessing route coordinates or speed physics.
- Grounding & Zero Hallucination (RAG First):** Every regulatory guideline, hazardous cargo rule, and driver policy returned by the AI is strictly bound to citations from the enterprise knowledge base, following the RAG paradigm (*arXiv:2005.11401*).
- Human-in-the-Loop Dispatch Control:** The AI optimizes and proposes; the human dispatcher retains authority to review, adjust, override, and approve dispatches. All agent-recommended changes require explicit human confirmation.
- Auditability & Traceability:** Every state change, manual override, re-route calculation, and agent decision is permanently logged in an immutable audit trail for compliance and post-trip analysis.
- Sub-Second Responsiveness & Intuitive UX:** Complex backend logistics operations are abstracted into a clean, modern interface built with Next.js, React Query, and Tailwind CSS, giving operators immediate clarity under high-pressure dispatch environments.

Implementation Milestones

Milestone 1: Setup & Foundations

- Initialize Next.js (TypeScript, Tailwind CSS, React Query) frontend repository.
- Setup FastAPI backend with PostgreSQL schema, migration tooling (Alembic), and JWT authentication.
- Establish core database models: Users, Hubs, Vehicles, Drivers, Shipments, and Routes.

Milestone 2: Core Features & Route Optimization

- Implement full CRUD APIs and management views for fleet assets and shipments.
- Develop DSA/VRPTW route optimization engine with capacity and time window constraints.
- Integrate interactive map visualization with route polylines and stop sequencing.
- Build live trip tracking, status transition state machine, and trip completion workflows.

Milestone 3: AI Copilot, RAG & Production Deployment

- Implement LangGraph multi-agent orchestration for natural language dispatch assistance and re-routing.
- Implement RAG pipeline with vector embeddings over logistics SOPs, safety documents, and the WikiQA dataset.
- Build the operational analytics & KPI dashboard (fuel, distance, SLA adherence).
- Containerize using Docker, configure CI/CD pipelines, and deploy the working application.